

FP&A Financeiro

Documentacao Tecnica

Plataforma interna de planejamento orcamentario e acompanhamento Realizado x Orcado, integrada ao ERP System SIA.

README	Visao geral, stack, setup e deploy
API	Referencia completa de endpoints
Banco de Dados	Schema, tabelas, views e migrations
ETL	Pipeline SIA -> DW
Queries	Todas as queries: ETL, DW, API e Views
Decisooes Tecnicas	Architecture Decision Records

Gerado em 26/05/2026

README

Visao geral, stack, setup e deploy

FP&A Financeiro — Plataforma de Planejamento Orçamentário

Plataforma interna para orçamento empresarial e acompanhamento **Realizado x Orçado**, integrada ao ERP System SIA via ETL.

Stack

Camada	Tecnologia
Backend	Python 3.11 · FastAPI · SQLAlchemy 2 · Alembic
Banco analítico	PostgreSQL 15 (schemas dw e app)
ETL	Python · pandas · pyodbc (Firebird ODBC)
Frontend	React 18 · TypeScript · Vite · Recharts · TanStack Query v5
BI	Metabase v0.50 (conectado ao schema dw)
Deploy	Docker Compose · Nginx

Funcionalidades

- * **Orçamento** — lançamento por conta gerencial, centro de custo e mês; versionamento (Original, Revisão, Forecast)
- * **Comparativo Realizado x Orçado** — por versão, empresa e período
- * **DRE Gerencial** — hierarquia de contas com realizado e orçado
- * **Workflow de Aprovação** — RASCUNHO → ENVIADO → APROVADO | REPROVADO; notificações SMTP
- * **Mapeamentos** — contas SIA → gerenciais; CCs SIA → gerenciais
- * **Metabase** — 5 views analíticas prontas para dashboard

Pré-requisitos

- * Docker + Docker Compose v2
- * Python 3.11+ (desenvolvimento local)
- * Node.js 20+ (desenvolvimento local)
- * Driver ODBC Firebird instalado no servidor do ETL

Setup — Desenvolvimento local

1. Variáveis de ambiente

```
cp .env.example .env
# Preencha .env com as credenciais reais – nunca commitar!
```

Variáveis obrigatórias:

```
DW_HOST / DW_PORT / DW_NAME / DW_USER / DW_PASSWORD
DATABASE_URL
SIA_HOST / SIA_PORT / SIA_DATABASE / SIA_USER / SIA_PASSWORD / SIA_CODEMP
VITE_API_URL=http://localhost:8000/api/v1
VITE_EMPRESA_CODEMP=1 # EMP_COD da empresa principal no SIA
```

2. Banco de dados (dev)

```
docker compose -f docker-compose.dev.yml up -d
```

3. Backend

```
cd api
python -m venv .venv
.venv\Scripts\activate # Windows
# source .venv/bin/activate # Linux/macOS
pip install -r requirements.txt

cd ..
alembic upgrade head # aplica todas as migrations

cd api
uvicorn api.main:app --reload --port 8000
# Docs interativa: http://localhost:8000/docs
```

4. Frontend

```
cd frontend
cp .env.example .env # ou crie com VITE_API_URL e VITE_EMPRESA_CODEMP
npm install
npm run dev
# http://localhost:5173
```

5. ETL (manual)

```
cd etl
pip install -r requirements.txt
python pipeline.py --ano 2025 --mes 1
# Opções: --ano, --mes, --codemp (forçar empresa)
```

Deploy — Produção

```
# 1. Build do frontend
cd frontend && npm run build && cd ..

# 2. Subir todos os serviços
docker compose up -d --build
```

Serviço	URL
Frontend	http://servidor/
API REST	http://servidor/api/v1/
Swagger	http://servidor/docs
Metabase	http://servidor/metabase/

Migrations

```
alembic upgrade head # aplicar todas
alembic downgrade -1 # reverter uma
alembic revision --autogenerate -m "descr" # gerar nova
```

Sequência atual: 001 → 002 → 003 → 004

Migration	Descrição
001	Schema inicial — todas as tabelas do DW
002	Fix dim_conta_sia (codemp→codpla, adiciona conta_class); remove codemp de dim_fornecedor
003	5 views analíticas para Metabase
004	Corrige v_comparativo_mensal — remove join por CC (todos os lançamentos SIA têm MOV_CECT = NULL)

Estrutura de Pastas

```

financeiro-fpa/
├── api/
│   ├── config.py # Settings via pydantic-settings (.env)
│   ├── main.py # FastAPI app + CORS + routers
│   ├── db/ # Engine, session, Base
│   ├── models/ # ORM: dimensoes, fatos, workflow, mapeamento
│   ├── schemas/ # Pydantic I/O schemas
│   ├── routers/ # 9 routers REST
│   ├── services/
│   └── email.py # Notificações SMTP (stdlib smtplib)
├── etl/
│   ├── pipeline.py # Orquestrador: 4 passos por execução
│   ├── extractor.py # SIA read-only via pyodbc
│   ├── transformer.py # Normalização, Decimal, chave de idempotência
│   ├── loader.py # Upserts idempotentes no DW
│   ├── config.py # ETLConfig (dataclass)
│   └── queries/ # SQL separado por módulo (ctb, ger, crc, cpq, fis)
├── frontend/src/
│   ├── pages/ # 8 páginas React
│   ├── hooks/ # React Query hooks (5 arquivos)
│   ├── services/
│   │   ├── api.ts # Axios client + todos os tipos TS + funções HTTP
│   │   └── format.ts # formatCurrency, formatPercent
│   ├── components/ # Layout, KpiCard, etc.
│   └── migrations/
├── versions/ # 001-004
├── infra/
│   ├── nginx/nginx.conf
│   ├── postgres/init.sql # Cria roles (fpa_user, metabase_reader)
│   ├── metabase/ # SETUP.md + SQLs prontos para Metabase
│   └── docs/ # Documentação técnica detalhada
├── API.md
├── BANCO_DE_DADOS.md
├── ETL.md
├── DECISOES_TECNICAS.md
├── .env.example
├── docker-compose.yml # Produção (db, api, etl, metabase, nginx)
├── docker-compose.dev.yml # Dev (só db)
├── alembic.ini
└── CLAUDE.md # Contexto para Claude Code

```

Endpoints da API

Documentação completa em docs/API.md. Resumo:

Método	Rota	Descrição
GET	/health	Health check
GET	/api/v1/empresas/	Lista empresas
GET	/api/v1/centros-custo/	Lista CCs gerenciais
POST	/api/v1/centros-custo/	Cria CC
GET	/api/v1/contas-gerenciais/	Lista plano gerencial
POST	/api/v1/contas-gerenciais/	Cria conta
GET	/api/v1/versoes-orcamento/{ano}	Lista versões

Método	Rota	Descrição
POST	/api/v1/versoes-orcamento/	Cria versão
GET	/api/v1/orcamento/{ano}/{id_versao}	Carrega orçamento
POST	/api/v1/orcamento/	Salva célula (upsert)
GET	/api/v1/comparativo/{ano}/{id_versao}	Realizado × Orçado
GET	/api/v1/dre/{ano}/{id_versao}	DRE gerencial
GET	/api/v1/lancamentos/{YYYY-MM}	Lançamentos realizados
GET	/api/v1/workflow/	Lista workflows
POST	/api/v1/workflow/iniciar	Cria RASCUNHO
POST	/api/v1/workflow/{id}/enviar	RASCUNHO → ENVIADO
POST	/api/v1/workflow/{id}/aprovar	ENVIADO → APROVADO
POST	/api/v1/workflow/{id}/reprovar	ENVIADO → REPROVADO
GET	/api/v1/mapeamentos/contas	Lista mapeamentos de contas
POST	/api/v1/mapeamentos/contas	Cria mapeamento
GET	/api/v1/mapeamentos/centros-custo	Lista mapeamentos de CC
POST	/api/v1/mapeamentos/centros-custo	Cria mapeamento de CC

Regras absolutas

- > **Nunca float** — `Decimal` no Python, `NUMERIC(15,2)` no PostgreSQL
- > **SIA é read-only** — zero INSERT/UPDATE/DELETE no banco Firebird
- > **Secrets no .env** — nunca commitar senhas ou tokens
- > **ETL idempotente** — `ON CONFLICT DO UPDATE` em todos os upserts
- > **Filtrar por empresa** — queries no SIA sempre usam `CODEMP` ou equivalente
- > **Escala monetária SIA** — `MOV_VALOR` é `NUMERIC` nativo no Firebird, **não dividir por escala**

Testes

```
# Backend
cd api && pytest tests/ -v

# ETL
cd etl && pytest tests/ -v
```

Documentação adicional

Documento	Conteúdo
docs/API.md	Todos os endpoints com parâmetros e exemplos
docs/BANCO_DE_DADOS.md	Schema completo, tabelas, índices, views
docs/ETL.md	Pipeline ETL passo a passo, colunas SIA validadas
docs/DECISOES_TECNICAS.md	ADRs, limitações do SIA, estratégias adotadas
infra/metabase/SETUP.md	Configuração do Metabase e dashboards
CLAUDE.md	Contexto completo para o Claude Code

API - Referencia Completa

Todos os endpoints com parametros e exemplos

API — Referência Completa

Base URL: <http://servidor/api/v1> Documentação interativa (Swagger): <http://servidor/docs>

Todos os valores monetários são retornados como `string` numérica com precisão de 2 casas decimais (NUMERIC do PostgreSQL serializado via Pydantic).

Sistema

GET /health

Retorna status da API.

```
{ "status": "ok", "env": "production" }
```

Empresas

GET /empresas/

Lista empresas cadastradas (espelho de GER_EMPRESAS do SIA).

Parâmetro	Tipo	Padrão	Descrição
apenas_ativas	bool	true	Filtrar só ativas

Resposta:

```
[  
  { "id": 1, "codemp": 1, "nome": "Philozon", "ativa": true },  
  { "id": 2, "codemp": 3, "nome": "Philozon", "ativa": true }  
]
```

Nota: id é o *autoincrement* do DW. codemp é o EMP_COD no SIA. O frontend resolve o id correto via `VITE_EMPRESA_CODEMP → useEmpresaAtiva()`. Nunca *hardcodar* `id_empresa = 1`.

Centros de Custo

GET /centros-custo/

Parâmetro	Tipo	Padrão
apenas_ativos	bool	true

```
[{ "id": 1, "codigo": "CC01", "nome": "Comercial", "ativo": true, "id_pai": null }]
```

POST /centros-custo/

```
{ "codigo": "CC02", "nome": "Administrativo", "descricao": null, "id_pai": null }
```

PUT /centros-custo/{id}

Campos opcionais: nome, descricao, ativo.

DELETE /centros-custo/{id}

Soft delete — define ativo = false.

Contas Gerenciais

GET /contas-gerenciais/

Parâmetro	Tipo	Padrão	
tipo	string	—	Filtrar por tipo (RECEITA, DESPESA, ...)
apenas_ativas	bool	true	

```
[{
  "id": 5,
  "codigo": "3.01.01",
  "nome": "Receita Bruta de Vendas",
  "tipo": "RECEITA",
  "natureza": "CREDORA",
  "nivel": 3,
  "id_pai": 3,
  "aceita_lancamento": true,
  "ativa": true
}]
```

POST /contas-gerenciais/

Campos obrigatórios: codigo, nome, tipo, natureza. tipo ∈ {RECEITA, DESPESA, ATIVO, PASSIVO, RESULTADO} natureza ∈ {DEVEDORA, CREDORA}

PUT /contas-gerenciais/{id} / DELETE /contas-gerenciais/{id}

Versões de Orçamento

GET /versoes-orcamento/{ano}

```
[{
  "id": 1,
  "ano": 2025,
  "tipo": "ORIGINAL",
  "nome": "Orçamento Original 2025",
  "descricao": null,
  "data_criacao": "2025-01-01",
  "bloqueada": false
}]
```

POST /versoes-orcamento/

```
{
  "ano": 2025,
  "tipo": "ORIGINAL",
  "nome": "Orçamento Original 2025",
  "descricao": "Primeira versão aprovada"
}
```

tipo ☐ {ORIGINAL, REVISAO, FORECAST}

Orçamento

GET /orcamento/{ano}/{id-versao}

Parâmetro	Tipo	Descrição
id_empresa	int	Filtrar por empresa
id_centro_custo	int	Filtrar por CC

Retorna lista de células do orçamento (uma por conta × CC × mês).

POST /orcamento/

Cria ou atualiza uma célula (upsert por id_empresa + id-versao + id_conta_gerencial + id_centro_custo + ano + mes).

```
{
  "id_empresa": 1,
  "id-versao": 1,
  "id_conta_gerencial": 5,
  "id_centro_custo": 2,
  "ano": 2025,
  "mes": 1,
  "valor": "15000.00",
  "observacao": null
}
```

Retorna 409 se a versão estiver bloqueada.

Comparativo Realizado × Orçado

GET /comparativo/{ano}/{id-versao}

Parâmetro	Tipo	Descrição
id_empresa	int	Recomendado. Filtra orçado e realizado pela empresa. Sem filtro = visão consolidada de todas as empresas.
id_centro_custo	int	Filtra o orçado pelo CC (realizado sempre cobre o total, pois SIA não registra CC nos lançamentos)

Resposta:

```
{
  "ano": 2025,
  "id-versao": 1,
  "nome-versao": "Orçamento Original 2025",
  "itens": [
    {
      "mes": 1,
      "conta_gerencia_codigo": "3.01.01",
      "conta_gerencia_nome": "Receita Bruta",
      "centro_custo_codigo": null,
      "centro_custo_nome": null,
      "valor_orcado": "15000.00",
      "valor_realizado": "12500.50",
      "variacao_absoluta": "-2499.50",
      "variacao_percentual": "-16.66"
    }
  ],
  "total_orcado": "180000.00",
  "total_realizado": "150006.00",
  "variacao_absoluta_total": "-29994.00",
  "variacao_percentual_total": "-16.66"
}
```

***Por que** centro_custo_codigo/nome é null? Todos os lançamentos contábeis no SIA da Philozon têm MOV_CECT = NULL — o sistema não registra CC nos lançamentos. O realizado é, portanto, agregado apenas por conta + empresa + período. O orçado é somado em todos os CCs para a mesma conta-mês. Ver DECISOES_TECNICAS.md.*

DRE Gerencial

GET /dre/{ano}/{id-versao}

Parâmetro	Tipo	Descrição
id_empresa	int	Filtrar por empresa

```
{
  "ano": 2025,
  "id_versao": 1,
  "linhas": [
    {
      "id": 1,
      "codigo": "3",
      "nome": "RECEITAS",
      "tipo": "RECEITA",
      "natureza": "CREDORA",
      "nivel": 1,
      "id_pai": null,
      "valor_orcado": "180000.00",
      "valor_realizado": "150006.00"
    }
  ]
}
```

Retorna todas as contas gerenciais ativas com seus totais anuais (orçado e realizado). A hierarquia é resolvida no frontend via `id_pai`.

Lançamentos Realizados

GET /lançamentos/{mes_referencia}

Formato: YYYY-MM (ex: 2025-01)

Parâmetro	Tipo	Descrição
id_empresa	int	Filtrar por empresa
id_conta_gerencial	int	Filtrar por conta gerencial

Retorna até 500 lançamentos (uso diagnóstico / auditoria).

Mapeamentos

GET /mapeamentos/contas

Parâmetro	Tipo
id_empresa	int

```
[{
  "id": 1,
  "id_conta_sia": 42,
  "id_conta_gerencia": 5,
  "id_empresa": 1,
  "ativo": true,
  "conta_sia": {
    "id": 42,
    "conta_codigo": "312",
    "conta_class": "3.01.01",
    "conta_nome": "RECEITA BRUTA DE VENDAS"
  }
}]
```

POST /mapeamentos/contas

```
{ "id_conta_sia": 42, "id_conta_gerencia": 5, "id_empresa": 1 }
```

Retorna 409 se já existir mapeamento ativo para a conta SIA.

DELETE /mapeamentos/contas/{id} — **desativa (soft delete)**

GET /mapeamentos/centros-custo

```
[{
  "id": 1,
  "cc_sia_codigo": "10",
  "cc_sia_nome": "Vendas",
  "id_empresa": 1,
  "id_centro_custo_gerencia": 2,
  "ativo": true
}]
```

POST /mapeamentos/centros-custo

```
{ "cc_sia_codigo": "10", "cc_sia_nome": "Vendas", "id_empresa": 1,
  "id_centro_custo_gerencia": 2 }
```

DELETE /mapeamentos/centros-custo/{id} — **desativa**

Workflow de Aprovação

GET /workflow/

Parâmetro	Tipo	Descrição
ano	int	Filtrar por ano da versão

```
[{
  "id": 1,
  "id-versao": 1,
  "id-empresa": 1,
  "versao_nome": "Orçamento Original 2025",
  "versao_ano": 2025,
  "empresa_nome": "Philozon",
  "status": "ENVIADO",
  "criado_por": "Ana Silva",
  "enviado_por": "Ana Silva",
  "aprovado_por": null,
  "reprovado_por": null,
  "data_envio": "2025-02-10T09:30:00",
  "data_decisao": null,
  "comentario": null,
  "criado_em": "2025-02-08T14:00:00",
  "atualizado_em": "2025-02-10T09:30:00"
}]
```

GET /workflow/{id} — **detalhe**

POST /workflow/iniciar

Cria registro em status **RASCUNHO**. Retorna 409 se já existir workflow ativo (**RASCUNHO** ou **ENVIADO**) para a mesma versão+empresa.

```
{ "id-versao": 1, "id-empresa": 1, "criado_por": "Ana Silva" }
```

POST /workflow/{id}/enviar

Transição **RASCUNHO** → **ENVIADO**. Dispara e-mail para aprovadores em background.

```
{ "enviado_por": "Ana Silva" }
```

POST /workflow/{id}/aprovar

Transição **ENVIADO** → **APROVADO**. Bloqueia a versão (bloqueada = true). Dispara e-mail.

```
{ "aprovado_por": "Carlos Gestor", "comentario": "Aprovado conforme reunião." }
```

POST /workflow/{id}/reprovar

Transição **ENVIADO** → **REPROVADO**. Dispara e-mail.

```
{ "reprovado_por": "Carlos Gestor", "comentario": "Reduzir despesas de TI em 10%." }
```

O campo **comentario** é **obrigatório** ao reprovar.

GET /workflow/{id}/justificativas — **lista justificativas de variação**

POST /workflow/{id}/justificativas

```
{
  "id_empresa": 1,
  "id_versao": 1,
  "id_conta_gerencial": 5,
  "id_centro_custo": 2,
  "ano": 2025,
  "mes": 3,
  "valor_orcado": "15000.00",
  "valor_realizado": "18500.00",
  "variacao_absoluta": "3500.00",
  "variacao_percentual": "23.33",
  "justificativa": "Campanha de lançamento do produto X.",
  "criado_por": "Ana Silva"
}
```

Códigos de erro

Código	Significado
404	Recurso não encontrado
409	Conflito — duplicata ou estado inválido (ex: versão bloqueada, workflow já ativo)
422	Dados inválidos (validação Pydantic)
500	Erro interno — verifique logs da API

Autenticação

A API não implementa autenticação na versão atual — uso exclusivo em rede interna. Em produção, coloque o Nginx na frente com autenticação básica ou VPN.

Schema completo: tabelas, colunas, views e migrations

PostgreSQL 15. Dois schemas: `dw` (Data Warehouse — dimensões, fatos, workflow, mapeamentos) e `app` (reservado para uso futuro).

Todos os valores monetários usam `NUMERIC(15,2)`. Nunca `FLOAT` ou `REAL`.

dw.dim empresa

Espejo das empresas ativas do SIA (GER EMPRESAS). Populada pelo ETL.

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	Autoincrement interno do DW
codemp	INTEGER UNIQUE	NO	EMP_COD no SIA — chave de negócio
nome	VARCHAR(200)	NO	Nome da empresa
cnpj	VARCHAR(18)	YES	CNPJ formatado
ativa	BOOLEAN	NO	EMP_ATIINA = 'A' no SIA

Empresas Philozon conhecidas:

	Nome
1	Philozon
2	O3R
3	Philozon
4	Philozon
100	Inativa

Nunca hardcodar id_empresa = 1. O id interno do DW é atribuído por autoincrement e pode diferir do codemp. O frontend resolve via VITE_EMPRESA_CODEMP → useEmpresaAtiva().

dw.dim_tempo

Calendário diário. Gerado pelo transformer, não extraído do SIA.

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
data	DATE UNIQUE	NO	Data completa
ano	SMALLINT	NO	
mes	SMALLINT	NO	1–12
trimestre	SMALLINT	NO	1–4
semestre	SMALLINT	NO	1–2
nome_mes	VARCHAR(20)	NO	Ex: "Janeiro"

dw.dim_centro_custo

Centros de custo **gerenciais** — cadastro interno, independente do SIA. Gerenciados via API.

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
codigo	VARCHAR(50) UNIQUE	NO	Código gerencial (ex: "CC01")
nome	VARCHAR(200)	NO	
descricao	TEXT	YES	
id_pai	INTEGER FK	YES	Self-referência para hierarquia
ativo	BOOLEAN	NO	Soft delete

dw.dim_conta_gerencial

Plano de contas **gerencial** — separado do plano contábil do SIA. Cadastro interno, gerenciado via API.

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
codigo	VARCHAR(50) UNIQUE	NO	Ex: "3.01.01"
nome	VARCHAR(200)	NO	Ex: "Receita Bruta de Vendas"
tipo	VARCHAR(30)	NO	RECEITA, DESPESA, ATIVO, PASSIVO, RESULTADO
natureza	VARCHAR(10)	NO	DEVEDORA ou CREDORA
id_pai	INTEGER FK	YES	Self-referência para hierarquia
nivel	SMALLINT	NO	1 = raiz, 2 = grupo, 3 = folha
aceita_lancamento	BOOLEAN	NO	false para contas-grupo (só agrupam)
ativa	BOOLEAN	NO	Soft delete

dw.dim_conta_sia

Plano de contas contábil extraído do SIA (CTB_CONTAS). Somente leitura — nunca alterar manualmente.

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
codpla	INTEGER	NO	CON_CODPLA — plano de contas (Philozon usa 1 e 2)
conta_codigo	VARCHAR(50)	NO	CON_COD — código inteiro do SIA
conta_class	VARCHAR(50)	YES	CON_CLASS — código hierárquico legível (ex: "3.01.01")
conta_codsup	INTEGER	YES	CON_CODSUP — conta pai
conta_nome	VARCHAR(200)	NO	CON_DESC
conta_tipo	VARCHAR(10)	YES	CON_TIPO
conta_nivel	SMALLINT	YES	CON_NIVEL

CTB_CONTAS não tem CODEMP. O isolamento por empresa é feito via CON_CODPLA (planos 1 e 2 = Philozon).

dw.dim_versao_orcamento

Versões do orçamento anual.

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
ano	SMALLINT	NO	Ano de referência
tipo	VARCHAR(20)	NO	ORIGINAL, REVISAO, FORECAST
nome	VARCHAR(100)	NO	Nome descritivo
descricao	TEXT	YES	
data_criacao	DATE	NO	
bloqueada	BOOLEAN	NO	true após aprovação — impede edição do orçamento

dw.dim_cliente / dw.dim_fornecedor

Auxiliares para análises futuras de receita e despesa.

dim_cliente: extraído de GER_CLIDEST. Campos: codemp, cod_sia, nome, cnpj_cpf, ativo.

dim_fornecedor: extraído de GER_EMITENTES. **Não tem** CODEMP — cadastro global. Campos: cod_sia (EMI_COD), nome (EMI_DESC), nome_fantasia, cnpj_cpf, ativo.

Tabelas de Fato

dw.fato_lancamento_realizado

Lançamentos contábeis realizados, extraídos de CTB_MOVIMENTOS via ETL. **Nunca inserir manualmente.**

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
id_empresa	INTEGER FK	NO	→ dim_empresa
id_tempo	INTEGER FK	NO	→ dim_tempo
id_conta_sia	INTEGER FK	NO	→ dim_conta_sia
id_conta_gerencia	INTEGER FK	YES	→ dim_conta_gerencia (NULL se não mapeado)
id_centro_custo	INTEGER FK	YES	→ dim_centro_custo (SEMPRE NULL — MOV_CECT = NULL no SIA)

Coluna	Tipo	Nullable	Descrição
sia_lancamento_id	VARCHAR(100) UNIQUE	NO	Chave de idempotência: CODEMP_DATA_NUMLAN_CO DCON_TIPO
valor	NUMERIC(15,2)	NO	Valor absoluto (sem sinal)
tipo_lancamento	VARCHAR(1)	NO	D = Débito, C = Crédito
historico	VARCHAR(500)	YES	MOV_HIST
data_referencia	DATE	NO	MOV_DATA
data_carga	TIMESTAMP	NO	Preenchida automaticamente

id_centro_custo é sempre NULL porque MOV_CECT = NULL em todos os lançamentos contábeis do SIA da Philozon. Isso é uma característica do uso do sistema, não um bug. Todas as queries de comparativo devem agregar por (conta, empresa, período) ignorando CC.

dw.fato_orcamento

Orçamento inserido pelos usuários via frontend/API. Upsert por chave composta.

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
id_empresa	INTEGER FK	NO	→ dim_empresa
id-versao	INTEGER FK	NO	→ dim-versao-orcamento
id_conta_gerencia	INTEGER FK	NO	→ dim_conta_gerencia
id_centro_custo	INTEGER FK	NO	→ dim_centro_custo
ano	SMALLINT	NO	
mes	SMALLINT	NO	1–12
valor	NUMERIC(15,2)	NO	
observacao	TEXT	YES	
criado_por	VARCHAR(100)	YES	Nome do usuário
criado_em	TIMESTAMP	NO	
atualizado_em	TIMESTAMP	NO	Auto-updated

Chave de upsert (único lógico): (id_empresa, id-versao, id_conta_gerencia, id_centro_custo, ano, mes)

dw.fato_receita / dw.fato_despesa

Tabelas de fato auxiliares para receita bruta e despesas agregadas.

fato_receita: fonte FIS_MOVIMENTO. Campos: receita_bruta, deducoes, receita_liquida. Chave: chave_upsert = empresa+ano+mes+cliente.

fato_despesa: fonte CTB_MOVIMENTOS + mapeamento gerencial. Agrega por conta + CC. Chave: chave_upsert.

Tabelas Operacionais

dw.workflow_orcamento

Ciclo de aprovação do orçamento por versão/empresa.

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
id-versao	INTEGER FK	NO	→ dim-versao-orcamento
id-empresa	INTEGER FK	NO	→ dim-empresa
status	VARCHAR(20)	NO	RASCUNHO, ENVIADO, APROVADO, REPROVADO
criado_por	VARCHAR(100)	NO	
enviado_por	VARCHAR(100)	YES	Preenchido na transição RASCUNHO→ENVIADO
aprovado_por	VARCHAR(100)	YES	Preenchido na transição ENVIADO→APROVADO
reprovado_por	VARCHAR(100)	YES	Preenchido na transição ENVIADO→REPROVADO
data-envio	TIMESTAMP	YES	
data-decisao	TIMESTAMP	YES	Aprovação ou reprovação
comentario	TEXT	YES	Obrigatório ao reprovar
criado_em	TIMESTAMP	NO	
atualizado_em	TIMESTAMP	NO	Auto-updated

Transições de estado:

RASCUNHO → ENVIADO (POST /workflow/{id}/enviar)
ENVIADO → APROVADO (POST /workflow/{id}/aprovar) – bloqueia dim-versao-orcamento.bloqueada = true
ENVIADO → REPROVADO (POST /workflow/{id}/reprovar) – comentario obrigatório

dw.justificativa_variacao

Justificativas textuais para variações acima de threshold configurado.

Coluna	Tipo	Nullable
id	SERIAL PK	NO
id_empresa	INTEGER FK	NO
id-versao	INTEGER FK	NO
id_conta_gerencia	INTEGER FK	NO
id-centro_custo	INTEGER FK	NO
ano_mes	INTEGER	NO
valor_orcado	NUMERIC(15,2)	NO
valor_realizado	NUMERIC(15,2)	NO
variacao_absoluta	NUMERIC(15,2)	NO
variacao_percentual	NUMERIC(8,2)	NO
justificativa	TEXT	NO
criado_por	VARCHAR(100)	NO
criado_em	TIMESTAMP	NO

`dw.mapeamento_conta_sia_gerencia`

De-para: conta contábil SIA → conta gerencial interna. Uma conta SIA mapeia para no máximo uma conta gerencial por empresa (restrição lógica via API — retorna 409 se já existir mapeamento ativo).

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
id_conta_sia	INTEGER FK	NO	→ dim_conta_sia
id_conta_gerencia	INTEGER FK	NO	→ dim_conta_gerencia
id_empresa	INTEGER FK	NO	→ dim_empresa
ativo	BOOLEAN	NO	Soft delete
observacao	TEXT	YES	

`dw.mapeamento_centro_custo_sia_gerencia`

De-para: centro de custo SIA → CC gerencial interno.

Coluna	Tipo	Nullable	Descrição
id	SERIAL PK	NO	
cc_sia_codigo	VARCHAR(50)	NO	CC_COD do SIA (como string)

Coluna	Tipo	Nullable	Descrição
cc_sia_nome	VARCHAR(200)	YES	CC_DESC — denormalizado para UI
id_empresa	INTEGER FK	NO	→ dim_empresa
id_centro_custo_gerencial	INTEGER FK	NO	→ dim_centro_custo
ativo	BOOLEAN	NO	Soft delete
observacao	TEXT	YES	

Na prática, o mapeamento de CC é irrelevante para o comparativo porque id_centro_custo é sempre NULL em fato_lancamento_realizado. O mapeamento existe para uso futuro ou se o SIA vier a registrar CC nos lançamentos.

Views Analíticas (schema dw)

Criadas pelas migrations 003 e 004. Disponíveis ao Metabase via role metabase_reader.

dw.v_lancamentos_detalhado

Lançamentos com todas as dimensões desnormalizadas (join com empresa, tempo, conta SIA, conta gerencial, CC).

Campos: id, sia_lancamento_id, data_referencia, ano, mes, nome_mes, trimestre, semestre, id_empresa, empresa, sia_conta_codigo, sia_conta_class, sia_conta_nome, id_conta_gerencial, conta_gerencial_codigo, conta_gerencial_nome, conta_tipo, id_centro_custo, cc_codigo, cc_nome, tipo_lancamento, valor, valor_liquido (valor × sinal D/C), historico, data_carga.

dw.v_comparativo_mensal

Realizado × Orçado por mês/conta/versão/empresa. **Corrigida na migration 004** para ignorar CC no join (todos os lançamentos têm id_centro_custo = NULL).

Estratégia:

- > realizado_agg agrega por (empresa, conta, ano, mes) sem CC
- > fato_orcamento é agrupado somando todos os CCs por (empresa, conta, versão, mes)
- > LEFT JOIN por (empresa, conta, ano, mes)

Campos: ano, mes, nome_mes, id-versao, versao_nome, versao_tipo, id_empresa, empresa_nome, id_conta_gerencial, conta_codigo, conta_nome, conta_tipo, conta_natureza, valor_orcado, valor_realizado, variacao_absoluta, variacao_percentual.

dw.v_evolucao_mensal

Totais mensais de realizado por tipo de conta (RECEITA, DESPESA, etc.) e empresa.

Campos: ano, mes, nome_mes, trimestre, id_empresa, empresa, conta_tipo, valor_realizado, qtd_lancamentos.

dw.v_dre_anual

DRE completa com hierarquia de contas, orçado e realizado por ano/versão/empresa.

Lógica: CROSS JOIN entre dim_conta_gerenciais × dim-versao-orcamento × dim-empresa, filtrado por (o.valor_orcado IS NOT NULL OR r.valor_realizado IS NOT NULL). Inclui conta pai (codigo_pai, nome_pai) para resolução de hierarquia.

Campos: id_conta_gerenciais, codigo, nome, tipo, natureza, nivel, id_pai, codigo_pai, nome_pai, id-versao, versao_nome, ano, id_empresa, empresa_nome, valor_orcado, valor_realizado, variacao_absoluta, variacao_percentual.

dw.v_workflow_resumo

Status dos workflows com nomes de versão e empresa desnormalizados. Inclui horas_para_decisao calculado.

Campos: id, status, id-versao, versao_nome, versao_ano, versao_tipo, versao_bloqueada, id_empresa, empresa_nome, criado_por, enviado_por, aprovado_por, reprovado_por, data_envio, data_decisao, comentario, criado_em, atualizado_em, horas_para_decisao.

Migrations

Sequência linear: 001 → 002 → 003 → 004

```
alembic upgrade head # aplicar todas
alembic downgrade -1 # reverter uma
alembic current # ver revisão atual
```

Revisão	Arquivo	Descrição
001	001_initial_schema.py	Schema inicial — todos os schemas (dw, app), todas as tabelas, roles (fpa_user, metabase_reader)
002	002_fix_dim_conta_sia_fornecedor.py	Fix dim_conta_sia: rename codemp→codpla, adiciona conta_class; remove codemp de dim_fornecedor
003	003_views_analiticas.py	5 views analíticas para Metabase
004	004_fix_comparativo_cc.py	Corrige v_comparativo_mensal — remove join por CC (todos os lançamentos SIA têm MOV_CECT = NULL)

Roles e Permissões

Role	Permissões
fpa_user	SELECT, INSERT, UPDATE, DELETE em todo o schema dw
metabase_reader	SELECT em todas as tabelas e views do schema dw

Criados em `infra/postgres/init.sql`. Renovados via `GRANT SELECT ON ALL TABLES IN SCHEMA dw TO metabase_reader` ao final de cada migration que cria novas views.

Índices relevantes

Tabela	Coluna(s)	Tipo
dim_empresa	codemp	UNIQUE
dim_tempo	data	UNIQUE
dim_centro_custo	codigo	UNIQUE
dim_conta_gerencia	codigo	UNIQUE
fato_lancamento_realizado	sia_lancamento_id	UNIQUE
fato_receita	chave_upsert	UNIQUE
fato_despesa	chave_upsert	UNIQUE

ETL

Pipeline SIA -> DW: extração, transformação e carga

ETL — Documentação Técnica

Pipeline de extração do ERP System SIA (Firebird) para o Data Warehouse PostgreSQL.

Visão Geral

```
SIA (Firebird, read-only)
↓ SIAExtractor.extrair_*() – queries SELECT via pyodbc
DataFrame pandas
↓ transformer.py – normalização, Decimal, chave de idempotência
Lista de dicts
↓ DWLoader.upsert_*() – INSERT ... ON CONFLICT DO UPDATE
PostgreSQL DW (schema dw)
```

Princípios:

- * SIA é **somente leitura** — zero escrita no Firebird
- * Idempotente — pode rodar N vezes sem duplicar dados
- * Campos monetários SIA são **NUMERIC** nativos — **não dividir por escala**
- * Queries transacionais filtram por **CODEMP**; **CTB_CONTAS** e **CTB_CCUSTOS** filtram por plano

Como Executar

```
cd etl
pip install -r requirements.txt # se ainda não instalado

# Carga de um mês específico
python pipeline.py --ano 2025 --mes 1

# Forçar empresa específica (sobrescreve SIA_CODEMP do .env)
python pipeline.py --ano 2025 --mes 1 --codemp 3

# Sem argumentos: usa ano/mês atual + empresa do .env
python pipeline.py
```

Variáveis de ambiente necessárias (.env):

```
SIA_HOST=<hostname>
SIA_PORT=3050
SIA_DATABASE=<caminho do .fdb>
SIA_USER=<usuario>
SIA_PASSWORD=<senha>
SIA_CODEMP=1 # EMP_COD da empresa principal

DW_HOST=localhost
DW_PORT=5432
DW_NAME=fpa
DW_USER=fpa_user
DW_PASSWORD=<senha>
```

Passos do Pipeline

Passo 1 — dim_tempo

Gerado localmente pelo transformer (sem acesso ao SIA). Preenche todos os dias do mês solicitado.

```
gerar_dim_tempo(data_inicio, data_fim) # → DataFrame com colunas: data, ano, mes,
trimestre, semestre, nome_mes
loader.upsert_dim_tempo(df_tempo) # ON CONFLICT (data) DO NOTHING
```

Passo 2 — dim_empresa

Extrai todas as empresas ativas do SIA e sincroniza com o DW.

Query SIA: `SELECT EMP_COD, EMP_NOM, EMP_NOMFANT, EMP_CNPJCPF, EMP_ATIINA FROM GER_EMPRESAS`

Transformação:

```
* ativa = (EMP_ATIINA == 'A')
* nome = EMP_NOM (fallback EMP_NOMFANT)
```

Upsert: `ON CONFLICT (codemp) DO UPDATE SET nome, cnpj, ativa`

Passo 3 — dim_conta_sia

Extrai o plano de contas contábil do SIA (Philozon usa planos 1 e 2).

Query SIA:

```
SELECT CON_CODPLA, CON_COD, CON_CODSUP, CON_CLASS, CON_NIVEL, CON_TIPO, CON_DESC
FROM CTB_CONTAS
WHERE CON_CODPLA IN (1, 2) -- planos Philozon 2019 e 2023
AND CON_INAT <> 'S' -- apenas ativas
```

CTB_CONTAS **não tem coluna** CODEMP. O isolamento por empresa é feito pelo plano de contas (CON_CODPLA).

Upsert: `ON CONFLICT (codpla, conta_codigo) DO UPDATE SET conta_class, conta_nome, conta_tipo, conta_nivel`

Passo 4 — fato_lancamento_realizado

Extrai lançamentos contábeis do período e resolve FKs para o DW.

4a. Extração

Query SIA:

```
SELECT M.MOV_CODEMP, M.MOV_NUMLAN, M.MOV_DATA, M.MOV_CODCON,
M.MOV_CECT, M.MOV_TIPO, M.MOV_VALOR, M.MOV_HIST
FROM CTB_MOVIMENTOS M
WHERE M.MOV_CODEMP = ? -- SIA_CODEMP do .env
AND EXTRACT(YEAR FROM M.MOV_DATA) = ?
AND EXTRACT(MONTH FROM M.MOV_DATA) = ?
AND M.MOV_TIPO IN (1, 2) -- apenas Débito e Crédito (excluindo encerramento/transferência)
```

Parâmetros posicionais (não dict — driver ODBC Firebird usa ?):

```
self._query(SQL_LANCAMENTOS, (self._cfg.sia_codemp, ano, mes))
```

4b. Transformação

```
transformar_lancamentos_contabeis(df)
```

- * MOV_VALOR → Decimal via `sia_decimal()` (sem divisão — já é NUMERIC nativo)
- * MOV_TIPO → `tipo_lancamento: 1='D', 2='C'`
- * MOV_CECT → `cc_sia_codigo: NULL` → string vazia
- * Chave de idempotência:

```
sia_lancamento_id = f"{MOV_CODEMP}_{MOV_DATA}_{MOV_NUMLAN}_{MOV_CODCON}_{MOV_TIPO}"
```

MOV_NUMLAN é sequencial por empresa/data, não global — por isso inclui todos os discriminadores.

4c. Resolução de FKs

```
_resolver_fks_lancamentos(df, loader, id_empresa_dw)
```

FK	Obrigatória	Comportamento se não resolvida
id_empresa	SIM	Linha descartada com <code>logger.warning</code>
id_tempo	SIM	Linha descartada
id_conta_sia	SIM	Linha descartada
id_conta_gerencia	NÃO	Inserido com NULL — conta ainda não mapeada
id_centro_custo	NÃO	Inserido com NULL — MOV_CECT é sempre NULL no SIA

4d. Upsert

```
loader.upsert_fato_lancamento_realizado(records)
# ON CONFLICT (sia_lancamento_id) DO UPDATE SET valor, tipo_lancamento, historico,
id_conta_gerencial, id_centro_custo, data_carga
```

Arquivos do ETL

```
etl/
■■■ pipeline.py – Orquestrador: 4 passos, logging, argparse
■■■ extractor.py – SIAExtractor: conexão read-only, queries por módulo
■■■ transformer.py – Normalização, Decimal, chave de idempotência, dim_tempo
■■■ loader.py – DWLoader: todos os upserts idempotentes
■■■ config.py – ETLConfig (dataclass lida via .env)
■■■ queries/ – SQL separado por módulo do SIA
■■■ ctb.py – CTB_MOVIMENTOS, CTB_CONTAS, CTB_CCUSTOS
■■■ ger.py – GER_EMPRESAS
■■■ crc.py – CRC_TITULO, CRC_TITULOPARC
■■■ cpg.py – CPG_TITULO, CPG_TITULOPARC
■■■ fis.py – FIS_MOVIMENTO
```

Mapeamento de Colunas SIA (validado 2026-05)

CTB_MOVIMENTOS — Lançamentos contábeis

Campo SIA	Tipo	Coluna DW	Observação
MOV_CODEMP	INT	id_empresa (FK)	Filtro obrigatório
MOV_NUMLAN	INT	parte de sia_lancamento_id	Sequencial por empresa/data
MOV_DATA	DATE	data_referencia	Competência
MOV_CODCON	INT	conta_sia_codigo	FK → CTB_CONTAS.CON_COD
MOV_CECT	INT/NULL	cc_sia_codigo	Sempre NULL na Philozon
MOV_TIPO	INT	tipo_lancamento	1→'D', 2→'C'; tipos 3 e 4 excluídos
MOV_VALOR	NUMERIC	valor	NUMERIC nativo — não dividir
MOV_HIST	VARCHAR	historico	Histórico livre

CTB_CONTAS — Plano de contas

Não tem CODEMP. Filtrar por CON_CODPLA IN (1, 2) para Philozon.

Campo SIA	Tipo	Coluna DW
CON_CODPLA	INT	codpla
CON_COD	INT	conta_codigo
CON_CODSUP	INT	conta_codsups
CON_CLASS	VARCHAR	conta_class
CON_NIVEL	INT	conta_nivel
CON_TIPO	CHAR(1)	conta_tipo
CON_DESC	VARCHAR	conta_nome
CON_INAT	CHAR(1)	filtro: <> 'S'

GER_EMPRESAS — Empresas

Campo SIA	Tipo	Coluna DW
EMP_COD	INT	codemp
EMP_NOM	VARCHAR	nome
EMP_NOMFANT	VARCHAR	fallback nome
EMP_CNPJCPF	VARCHAR	cnpj
EMP_ATIINA	CHAR(1)	ativa ('A' = ativo)

CTB_CCUSTOS — Centros de custo SIA

Não tem CODEMP. Filtrar por CC_CODCCPL = 3 (plano "Philozon & Ozoncare").

Campo SIA	Tipo	Descrição
CC_CODCCPL	INT	Plano de CC
CC_COD	INT	Código (chave parcial)
CC_CLASS	VARCHAR	Código hierárquico
CC_DESC	VARCHAR	Descrição
CC_INAT	CHAR(1)	'S' = inativo

Campos Monetários — Regra Crítica

```
def sia_decimal(valor) -> Decimal:
    """
    MOV_VALOR, TIT_VAL, TITPAR_VAL são NUMERIC nativos no Firebird.
    Não há divisão por 100 ou qualquer escala.
    """
    if valor is None or (isinstance(valor, float) and pd.isna(valor)):
        return Decimal("0")
    return Decimal(str(valor))
```

Mesmo que `pyodbc` retorne `float` em alguns drivers, `Decimal(str(valor))` preserva a precisão original. **Nunca dividir por 100.**

Idempotência

Cada tabela tem uma chave única que garante que rodar o pipeline duas vezes não duplica dados:

Tabela	Chave de conflito
dim_tempo	data
dim_empresa	codemp
dim_conta_sia	(codpla, conta_codigo)
fato_lancamento_realizado	sia_lancamento_id
fato_receita	chave_upsert
fato_despesa	chave_upsert

Todos os loaders usam `INSERT INTO ... ON CONFLICT (...) DO UPDATE SET ....`

Estratégia de Carga Incremental

O pipeline processa **um mês por vez**. Para carregar um ano completo:

```
for mes in 1 2 3 4 5 6 7 8 9 10 11 12; do
python pipeline.py --ano 2025 --mes $mes
done
```

Ou no Docker Compose, o serviço `etl` pode ser configurado com um cron interno para executar mensalmente.

Troubleshooting

"FK não resolvida" — lançamentos descartados

```
WARNING - 42 lançamentos descartados: FK não resolvida (empresa, data ou conta ausente no DW).
```

Causas possíveis:

- > `dim_empresa` não foi carregada (passo 2 falhou) — reexecutar sem `--codemp`
- > `dim_conta_sia` não tem a conta — a query `CTB_CONTAS` excluiu contas inativas (`CON_INAT = 'S'`) que têm lançamentos históricos
- > `dim_tempo` não tem a data — improvável (gerado localmente), mas pode ocorrer se o período for inconsistente

"Empresa CODEMP=X não encontrada no DW"

O passo 2 (`dim_empresa`) não carregou a empresa antes do passo 4. Verificar logs do passo 2.

Valores realizados zerados no comparativo

Se `fato_lancamento_realizado` está populado mas `GET /comparativo` retorna `valor_realizado = 0`:

- > Verificar se os mapeamentos estão cadastrados (`GET /mapeamentos/contas`)
- > Os lançamentos têm `id_conta_gerencial = NULL` → precisam do mapeamento de contas
- > Após cadastrar mapeamentos, reexecutar o ETL para atualizar os lançamentos existentes

Driver ODBC Firebird

O driver deve estar instalado no servidor onde o ETL roda:

- * Windows: Firebird ODBC Driver de <https://firebirdsql.org/>
- * Linux: pacote `libfbclient2` + ODBC driver

A connection string em `extractor.py` pode precisar de ajuste no nome do driver (`DRIVER={...}`) conforme o instalado.

Queries

Todas as queries do projeto: ETL (SIA), carga DW, API analítica e views

Queries — Referência Completa

Todas as queries do projeto, organizadas por origem, propósito e lógica de negócio.

Índice

- > [ETL — Extração do SIA (Firebird, read-only)](#1-etl--extração-do-sia)
- > [ETL — Carga no DW (PostgreSQL, upserts)](#2-etl--carga-no-dw)
- > [ETL — Resolução de FKs (lookups internos)](#3-etl--resolução-de-fks)
- > [API — Queries Analíticas (PostgreSQL)](#4-api--queries-analíticas)
- > [API — Queries ORM (CRUD)](#5-api--queries-orm)
- > [Views Analíticas (Metabase)](#6-views-analíticas)

1. ETL — Extração do SIA

Todas as queries abaixo são executadas via `SIAExtractor` usando `pyodbc` em modo **somente leitura**. Parâmetros posicionais usam `?` (padrão do driver ODBC Firebird — não usar dicionário nem `:nome`).

1.1 SQL_PLANO_CONTAS — `etl/queries/ctb.py`

Propósito: Extrair o plano de contas contábil da Philozon para popular `dim_conta_sia`.

```
SELECT
C.CON_CODPLA, -- plano de contas (1 = Philozon 2019, 2 = Philozon 2023)
C.CON_COD, -- código numérico da conta (chave interna do SIA)
C.CON_CODSUP, -- código da conta pai (para hierarquia)
C.CON_CLASS, -- código hierárquico legível (ex: "3.01.02")
C.CON_NIVEL, -- nível na hierarquia (1 = raiz, 2 = grupo, 3+ = folha)
C.CON_TIPO, -- tipo da conta (analítica, sintética etc.)
C.CON_DESC, -- descrição/nome da conta
C.CON_INAT -- 'S' = inativa (filtrada no WHERE)
FROM CTB_CONTAS C
WHERE C.CON_CODPLA IN (1, 2) -- apenas planos da Philozon
AND C.CON_INAT <> 'S' -- apenas contas ativas
ORDER BY C.CON_CODPLA, C.CON_CLASS
```

Por que `CON_CODPLA IN (1, 2)`? `CTB_CONTAS` não tem coluna `CODEMP`. O isolamento por empresa é feito pelo plano de contas. A Philozon usa o plano 1 (estrutura de 2019) e o plano 2 (estrutura de 2023 em diante).

Destino: `dim_conta_sia` via `loader.upsert_dim_conta_sia()`

1.2 SQL_CENTROS_CUSTO — etl/queries/ctb.py

Propósito: Extrair centros de custo do SIA para referência nos mapeamentos.

```
SELECT
CC.CC_CODCCPL, -- plano de CC (3 = "Philozon & Ozoncare")
CC.CC_COD, -- código numérico do CC
CC.CC_CODSUP, -- código do CC pai
CC.CC_CLASS, -- código hierárquico legível
CC.CC_NIVEL, -- nível hierárquico
CC.CC_TIPO, -- tipo do CC
CC.CC_DESC, -- descrição/nome do CC
CC.CC_INAT -- 'S' = inativo
FROM CTB_CCUSTOS CC
WHERE CC.CC_CODCCPL = 3 -- plano 3 = "Philozon & Ozoncare"
AND CC.CC_INAT <> 'S' -- apenas CCs ativos
ORDER BY CC.CC_CLASS
```

Importante: CTB_CCUSTOS não tem CODEMP, assim como CTB_CONTAS. O plano 3 cobre todas as empresas Philozon.

Nota de uso: Esta query está implementada mas o dado de CC no ETL é apenas informativo. Na prática, MOV_CECT = NULL em todos os lançamentos do SIA da Philozon, então o mapeamento de CC no fato não é utilizado.

1.3 SQL_LANCAMENTOS — etl/queries/ctb.py

Propósito: Extrair lançamentos contábeis de um período para popular fato_lancamento_realizado. Esta é a query central do ETL.

```
SELECT
M.MOV_CODEMP, -- empresa (chave obrigatória, igual ao parâmetro ?)
M.MOV_NUMLAN, -- número sequencial do lançamento (por empresa + data)
M.MOV_DATA, -- data de competência do lançamento
M.MOV_CODCON, -- código da conta contábil (FK -> CTB_CONTAS.CON_COD)
M.MOV_CECT, -- código do centro de custo (SEMPRE NULL na Philozon)
M.MOV_TIPO, -- tipo: 1=Débito, 2=Crédito (3=Encerramento e 4=Transf. excluídos)
M.MOV_VALOR, -- valor do lançamento – NUMERIC nativo, não dividir por 100
M.MOV_HIST -- histórico livre do lançamento
FROM CTB_MOVIMENTOS M
WHERE M.MOV_CODEMP = ? -- parâmetro 1: empresa
AND M.MOV_TIPO IN (1, 2) -- apenas Débito e Crédito
AND EXTRACT(YEAR FROM M.MOV_DATA) = ? -- parâmetro 2: ano
AND EXTRACT(MONTH FROM M.MOV_DATA) = ? -- parâmetro 3: mês
ORDER BY M.MOV_DATA, M.MOV_NUMLAN, M.MOV_CODCON
```

Por que excluir tipos 3 e 4?

- * Tipo 3 (encerramento de exercício): lançamentos de ajuste de encerramento, não representam transações reais

- * Tipo 4 (transferência entre contas): movimentação interna sem impacto na análise gerencial

MOV_VALOR: É NUMERIC nativo no Firebird. O pyodbc pode retornar como float dependendo do driver; o transformer.py converte para Decimal via sia_decimal() sem divisão.

Chave de idempotência gerada no transformer:

```
sia_lancamento_id = f"{MOV_CODEMP}_{MOV_DATA}_{MOV_NUMLAN}_{MOV_CODCON}_{MOV_TIPO}"
```

MOV_NUMLAN sozinho não é globalmente único — é sequencial por empresa/data. O composto garante unicidade global.

Destino: fato_lancamento_realizado via `loader.upsert_fato_lancamento_realizado()`

1.4 SQL_EMPRESAS — `etl/queries/ger.py`

Propósito: Espelhar o cadastro de empresas do SIA no DW.

```
SELECT
E.EMP_COD, -- código da empresa (chave de negócio)
E.EMP_NOM, -- razão social
E.EMP_NOMFANT, -- nome fantasia (usado como fallback se EMP_NOM vazio)
E.EMP_CNPJCPF, -- CNPJ
E.EMP_ATIINA -- 'A' = ativa, 'I' = inativa
FROM GER_EMPRESAS E
WHERE E.EMP_ATIINA = 'A' -- apenas empresas ativas
ORDER BY E.EMP_COD
```

Sem parâmetro de empresa — lista todas as empresas ativas do SIA de uma vez.

Destino: dim_empresa via `loader.upsert_dim_empresa()`

1.5 SQL_CLIENTES — `etl/queries/ger.py`

Propósito: Extrair cadastro de clientes por empresa.

```
SELECT
C.CLI_CODEMP, -- empresa (chave composta)
C.CLI_COD, -- código do cliente (chave composta)
C.CLI_DESC, -- razão social
C.CLI_FANT, -- nome fantasia
C.CLI_CNPJCPF, -- CNPJ/CPF
C.CLI_ATIINA -- booleano (diferente de GER_EMPRESAS que usa char)
FROM GER_CLIDEST C
WHERE C.CLI_CODEMP = ? -- parâmetro: empresa
ORDER BY C.CLI_COD
```

Destino: dim_cliente via `loader.upsert_dim_cliente()`

1.6 SQL_FORNECEDORES — `etl/queries/ger.py`

Propósito: Extrair cadastro global de fornecedores.

```
SELECT
E.EMI_COD, -- código do fornecedor (chave única — sem CODEMP)
E.EMI_DESC, -- razão social
E.EMI_FANT, -- nome fantasia
E.EMI_CNPJCPF, -- CNPJ/CPF
E.EMI_ATIINA -- booleano (ativo/inativo)
FROM GER_EMITENTES E
ORDER BY E.EMI_COD
```

GER_EMITENTES **não tem** CODEMP — é um cadastro global compartilhado por todas as empresas do SIA. Portanto nenhum filtro por empresa é aplicado.

Destino: `dim_fornecedor` via `loader.upsert_dim_fornecedor()`

1.7 SQL_CONTAS_RECEBER — [etl/queries/crc.py](#)

Propósito: Extrair títulos a receber com parcelas para análise de inadimplência e fluxo.

```
SELECT
T.TIT_CODEMP, -- empresa
T.TIT_LAN, -- número do título (chave: CODEMP + LAN)
T.TIT_CODCLI, -- código do cliente (FK -> GER_CLIDEST)
T.TIT_DOC, -- número do documento (NF, boleto etc.)
T.TIT_DTEMI, -- data de emissão do título
T.TIT_HIS, -- histórico/observação
TP.TITPAR_NUM, -- número da parcela
TP.TITPAR_DTVENC, -- data de vencimento da parcela
TP.TITPAR_VAL, -- valor da parcela – NUMERIC nativo
TP.TITPAR_SAL, -- saldo em aberto da parcela – NUMERIC nativo
TP.TITPAR_SIT -- situação: 'A'=Aberta, 'L'=Liquidada, etc.
FROM CRC_TITULO T
JOIN CRC_TITULOPARC TP
ON TP.TITPAR_CODEMP = T.TIT_CODEMP -- join pelo CODEMP (obrigatório)
AND TP.TITPAR_LANTIT = T.TIT_LAN -- join pelo número do título
WHERE T.TIT_CODEMP = ? -- parâmetro 1: empresa
AND EXTRACT(YEAR FROM TP.TITPAR_DTVENC) = ? -- parâmetro 2: ano
AND EXTRACT(MONTH FROM TP.TITPAR_DTVENC) = ? -- parâmetro 3: mês
ORDER BY TP.TITPAR_DTVENC, T.TIT_LAN, TP.TITPAR_NUM
```

Filtra pelo vencimento da parcela, não pela emissão do título. Isso dá a visão de fluxo de caixa do período.

Join seguro: O join sempre inclui `CODEMP` — se omitido, o join cruzaria parcelas de empresas diferentes.

1.8 SQL_CONTAS_PAGAR — [etl/queries/cpg.py](#)

Propósito: Extrair títulos a pagar com parcelas para análise de fluxo de saída.

```

SELECT
T.TIT_CODEMP, -- empresa
T.TIT_LAN, -- número do título
T.TIT_CODCRE, -- código do credor (FK -> GER_EMITENTES.EMI_COD)
T.TIT_DOC, -- número do documento (NF do fornecedor)
T.TIT_DTEMI, -- data de emissão
T.TIT_DTENT, -- data de entrada/recebimento da NF
T.TIT_HIS, -- histórico
TP.TITPAR_NUM, -- número da parcela
TP.TITPAR_DTVENC, -- data de vencimento
TP.TITPAR_VAL, -- valor da parcela – NUMERIC nativo
TP.TITPAR_SAL, -- saldo em aberto – NUMERIC nativo
TP.TITPAR_SIT -- situação da parcela
FROM CPG_TITULO T
JOIN CPG_TITULOPARC TP
ON TP.TITPAR_CODEMP = T.TIT_CODEMP
AND TP.TITPAR_LANTIT = T.TIT_LAN
WHERE T.TIT_CODEMP = ?
AND EXTRACT(YEAR FROM TP.TITPAR_DTVENC) = ?
AND EXTRACT(MONTH FROM TP.TITPAR_DTVENC) = ?
ORDER BY TP.TITPAR_DTVENC, T.TIT_LAN, TP.TITPAR_NUM

```

Estrutura idêntica a `SQL_CONTAS_RECEBER`, mas no módulo CPG. A diferença principal é `TIT_CODCRE` (credor/fornecedor) em vez de `TIT_CODCLI` (cliente).

1.9 SQL_RECEITAS — `etl/queries/fis.py`

Propósito: Extrair receita bruta de faturamento via documentos fiscais.

```

SELECT
M.MOV_CODEMP, -- empresa
M.MOV_NUMERO, -- número do documento fiscal
M.MOV_DATA, -- data de emissão
M.MOV_CLIFOR, -- código do cliente
M.MOV_VALORTOTAL, -- TODO: verificar se INT64 (÷100) ou NUMERIC nativo
M.MOV_VALORDESC, -- descontos
M.MOV_TIPO -- 'S' = saída (faturamento)
FROM FIS_MOVIMENTO M
WHERE M.MOV_CODEMP = ?
AND M.MOV_TIPO = 'S' -- apenas NFs de saída
AND EXTRACT(YEAR FROM M.MOV_DATA) = ?
AND EXTRACT(MONTH FROM M.MOV_DATA) = ?
ORDER BY M.MOV_DATA

```

ATENÇÃO: Colunas do `FIS_MOVIMENTO` ainda não foram validadas via MCP Firebird. O campo `MOV_VALORTOTAL` pode ser INT64 (necessitando divisão por 100) diferentemente do `CTB_MOVIMENTOS`. Validar antes de usar em produção.

2. ETL — Carga no DW

Queries executadas pelo `DWLoader` no PostgreSQL (schema `dw`). Todas usam `INSERT ... ON CONFLICT DO UPDATE` para garantir idempotência.

2.1 Upsert dim_tempo

```
INSERT INTO dw.dim_tempo (data, ano, mes, trimestre, semestre, nome_mes)
VALUES (:data, :ano, :mes, :trimestre, :semestre, :nome_mes)
ON CONFLICT (data) DO NOTHING
```

Conflito em: data (UNIQUE). **Ação:** DO NOTHING — datas já existentes são simplesmente ignoradas. Os dados de calendário não mudam.

2.2 Upsert dim_empresa

```
INSERT INTO dw.dim_empresa (codemp, nome, cnpj, ativa)
VALUES (:codemp, :nome, :cnpj, :ativa)
ON CONFLICT (codemp) DO UPDATE SET
nome = EXCLUDED.nome,
cnpj = EXCLUDED.cnpj,
ativa = EXCLUDED.ativa
```

Conflito em: codemp (UNIQUE). **Ação:** Atualiza todos os campos — mantém o cadastro sincronizado com o SIA. Se uma empresa for desativada no SIA, *ativa* será atualizado para *false*.

2.3 Upsert dim_conta_sia

```
INSERT INTO dw.dim_conta_sia
(codpla, conta_codigo, conta_class, conta_codsup, conta_nome, conta_tipo, conta_nivel)
VALUES
(:codpla, :conta_codigo, :conta_class, :conta_codsup, :conta_nome, :conta_tipo,
:conta_nivel)
ON CONFLICT (codpla, conta_codigo) DO UPDATE SET
conta_class = EXCLUDED.conta_class,
conta_codsup = EXCLUDED.conta_codsup,
conta_nome = EXCLUDED.conta_nome,
conta_tipo = EXCLUDED.conta_tipo,
conta_nivel = EXCLUDED.conta_nivel
```

Conflito em: (codpla, conta_codigo) — par plano + código é único. **Ação:** Atualiza todos os metadados da conta. O código *CON_CLASS* (hierárquico) pode mudar quando o plano é reestruturado.

2.4 Upsert dim_fornecedor

```
INSERT INTO dw.dim_fornecedor (cod_sia, nome, nome_fantasia, cnpj_cpf, ativo)
VALUES (:cod_sia, :nome, :nome_fantasia, :cnpj_cpf, :ativo)
ON CONFLICT (cod_sia) DO UPDATE SET
nome = EXCLUDED.nome,
nome_fantasia = EXCLUDED.nome_fantasia,
cnpj_cpf = EXCLUDED.cnpj_cpf,
ativo = EXCLUDED.ativo
```

Conflito em: cod_sia (EMI_COD — sem CODEMP pois é cadastro global).

2.5 Upsert fato_lancamento_realizado

```

INSERT INTO dw.fato_lancamento_realizado
(id_empresa, id_tempo, id_conta_sia, id_conta_gerencia, id_centro_custo,
sia_lancamento_id, valor, tipo_lancamento, historico, data_referencia)
VALUES
(:id_empresa, :id_tempo, :id_conta_sia, :id_conta_gerencia, :id_centro_custo,
:sia_lancamento_id, :valor, :tipo_lancamento, :historico, :data_referencia)
ON CONFLICT (sia_lancamento_id) DO UPDATE SET
valor = EXCLUDED.valor,
tipo_lancamento = EXCLUDED.tipo_lancamento,
historico = EXCLUDED.historico,
data_carga = now()

```

Conflito em: `sia_lancamento_id` (chave composta `CODEMP_DATA_NUMLAN_CODCON_TIPO`). **Ação:** Atualiza valor e histórico — permite corrigir lançamentos alterados no SIA. **Não atualiza:** `id_conta_gerencia` nem `id_centro_custo` — se o mapeamento mudar após a carga, é necessário reprocessar o período.

2.6 Upsert `fato_receita`

```

INSERT INTO dw.fato_receita
(id_empresa, id_tempo, id_cliente, chave_upsert,
receita_bruta, deducos, receita_liquida)
VALUES
(:id_empresa, :id_tempo, :id_cliente, :chave_upsert,
:receita_bruta, :deducos, :receita_liquida)
ON CONFLICT (chave_upsert) DO UPDATE SET
receita_bruta = EXCLUDED.receita_bruta,
deducos = EXCLUDED.deducos,
receita_liquida = EXCLUDED.receita_liquida,
data_carga = now()

```

Conflito em: `chave_upsert` (`empresa+ano+mes+cliente`).

3. ETL — Resolução de FKs

Queries de lookup executadas antes do upsert dos fatos, para transformar chaves do SIA em IDs do DW.

3.1 `resolver_ids_empresa`

```

SELECT id, codemp
FROM dw.dim_empresa
WHERE codemp = ANY(:vals)

```

Entrada: lista de `EMP_COD` do SIA. **Saída:** dicionário `{codemp → id}`.

3.2 `resolver_ids_tempo`

```
SELECT id, data::text
FROM dw.dim_tempo
WHERE data::text = ANY(:vals)
```

Entrada: lista de datas ISO (strings). **Saída:** dicionário {data_str → id}. O cast data::text permite comparação por ANY(:vals) com lista de strings.

3.3 resolver_ids_conta_sia

```
SELECT DISTINCT ON (conta_codigo) id, conta_codigo
FROM dw.dim_conta_sia
WHERE conta_codigo = ANY(:vals)
ORDER BY conta_codigo, codpla DESC
```

Entrada: lista de CON_COD (como strings). **Saída:** dicionário {conta_codigo → id}. DISTINCT ON (conta_codigo) com ORDER BY codpla DESC garante que, se o mesmo CON_COD existir em dois planos, o plano mais recente (maior codpla) tem prioridade.

3.4 resolver_mapeamentos_conta

```
SELECT cs.conta_codigo, m.id_conta_gerencial
FROM dw.mapeamento_conta_sia_gerencial m
JOIN dw.dim_conta_sia cs ON cs.id = m.id_conta_sia
WHERE m.id_empresa = :id_empresa
AND m.ativo = true
```

Entrada: id_empresa (DW). **Saída:** dicionário {conta_codigo → id_conta_gerencial}. Usado para popular id_conta_gerencial em fato_lancamento_realizado. Se a conta não tiver mapeamento, o campo fica NULL e o lançamento não aparece no comparativo gerencial.

3.5 resolver_mapeamentos_cc

```
SELECT cc_sia_codigo, id_centro_custo_gerencial
FROM dw.mapeamento_centro_custo_sia_gerencial
WHERE id_empresa = :id_empresa
AND ativo = true
```

Entrada: id_empresa (DW). **Saída:** dicionário {cc_sia_codigo → id_centro_custo_gerencial}. Na prática, sempre retorna vazio porque MOV_CECT = NULL em todos os lançamentos.

4. API — Queries Analíticas

Queries SQL raw executadas via sqlalchemy.text() nos routers da FastAPI. São as mais complexas do projeto.

4.1 Comparativo Realizado × Orçado — api/routers/comparativo.py

Endpoint: GET /comparativo/{ano}/{id-versao} **Propósito:** Principal query analítica. Junta orçamento (por conta+CC+mês) com realizado (por conta+mês, sem CC) para mostrar desvios.

```
WITH realizado_agg AS (  
  -- CTE 1: agrega realizado por (conta, ano, mes)  
  -- Ignora CC porque MOV_CECT = NULL em todos os lançamentos do SIA.  
  -- O filtro :id_empresa é aplicado AQUI dentro do CTE, não no JOIN externo,  
  -- para evitar dupla contagem multi-empresa quando agrupando sem empresa.  
  SELECT  
    id_conta_gerencial,  
    EXTRACT(YEAR FROM data_referencia)::int AS ano,  
    EXTRACT(MONTH FROM data_referencia)::int AS mes,  
    SUM(valor * CASE WHEN tipo_lancamento = 'D' THEN 1 ELSE -1 END)  
    AS valor_realizado  
  FROM dw.fato_lancamento_realizado  
  WHERE id_conta_gerencial IS NOT NULL  
  AND (:id_empresa IS NULL OR id_empresa = :id_empresa)  
  GROUP BY id_conta_gerencial,  
    EXTRACT(YEAR FROM data_referencia),  
    EXTRACT(MONTH FROM data_referencia)  
)  
SELECT  
  fo.mes,  
  cg.codigo AS conta_gerencial_codigo,  
  cg.nome AS conta_gerencial_nome,  
  NULL::varchar AS centro_custo_codigo, -- sempre NULL  
  NULL::varchar AS centro_custo_nome, -- sempre NULL  
  SUM(fo.valor) AS valor_orcado, -- soma todos os CCs  
  COALESCE(MAX(r.valor_realizado), 0) AS valor_realizado -- um por (conta,mes)  
FROM dw.fato_orcamento fo  
JOIN dw.dim_conta_gerencial cg ON cg.id = fo.id_conta_gerencial  
LEFT JOIN realizado_agg r  
  ON r.id_conta_gerencial = fo.id_conta_gerencial  
  AND r.ano = fo.ano  
  AND r.mes = fo.mes  
WHERE fo.ano = :ano  
  AND fo.id-versao = :id-versao  
  AND (:id_empresa IS NULL OR fo.id_empresa = :id_empresa)  
  AND (:id_centro_custo IS NULL OR fo.id_centro_custo = :id_centro_custo)  
GROUP BY fo.mes, cg.codigo, cg.nome  
ORDER BY fo.mes, cg.codigo
```

Parâmetros: :ano, :id-versao, :id_empresa (opcional), :id_centro_custo (opcional, só filtra orçamento).

Cálculo do valor_realizado:

- * tipo_lancamento = 'D' (Débito): multiplica por +1
- * tipo_lancamento = 'C' (Crédito): multiplica por -1
- * Para contas de despesa (natureza DEVEDORA): débitos são positivos — correto
- * Para contas de receita (natureza CREDORA): créditos são positivos (negativo × -1 = positivo) — correto

Por que MAX(r.valor_realizado) e não SUM? O CTE já agrupou sem empresa — há exatamente uma linha por (conta, mes) no realizado_agg. O MAX() é equivalente ao valor único; poderia ser MIN() também. O COALESCE trata contas sem realizado.

Decisão de design: Ver docs/DECIS0ES_TECNICAS.md ADR-003 e ADR-004.

4.2 DRE Gerencial — api/routers/dre.py

Endpoint: GET /dre/{ano}/{id-versao} **Propósito:** Retorna todas as contas gerenciais ativas com totais anuais de orçado e realizado. A hierarquia é resolvida no frontend via `id_pai`.

```
WITH orcado AS (  
  -- Total orçado anual por conta (soma todos os meses e CCs)  
  SELECT id_conta_gerencial, SUM(valor) AS total  
  FROM dw.fato_orcamento  
  WHERE ano = :ano AND id-versao = :id-versao  
  AND (:id_empresa IS NULL OR id_empresa = :id_empresa)  
  GROUP BY id_conta_gerencial  
) ,  
realizado AS (  
  -- Total realizado anual por conta (débitos positivos, créditos negativos)  
  SELECT id_conta_gerencial,  
  SUM(valor * CASE WHEN tipo_lancamento = 'D' THEN 1 ELSE -1 END) AS total  
  FROM dw.fato_lancamento_realizado  
  WHERE EXTRACT(YEAR FROM data_referencia) = :ano  
  AND (:id_empresa IS NULL OR id_empresa = :id_empresa)  
  AND id_conta_gerencial IS NOT NULL  
  GROUP BY id_conta_gerencial  
)  
SELECT  
  cg.id,  
  cg.codigo,  
  cg.nome,  
  cg.tipo,  
  cg.natureza,  
  cg.nivel,  
  cg.id_pai,  
  COALESCE(o.total, 0) AS valor_orcado,  
  COALESCE(r.total, 0) AS valor_realizado  
FROM dw.dim_conta_gerencial cg  
LEFT JOIN orcado o ON o.id_conta_gerencial = cg.id  
LEFT JOIN realizado r ON r.id_conta_gerencial = cg.id  
WHERE cg.ativa = true  
ORDER BY cg.codigo
```

Diferença em relação ao comparativo: O DRE retorna totais **anuais** (sem desagregar por mês) e **todas as contas ativas** (mesmo as sem dados). A hierarquia é plana no banco — o frontend usa `id_pai` para montar a árvore.

4.3 Lançamentos Realizados — `api/routers/lançamentos.py`

Endpoint: GET /lançamentos/{mes_referencia} **Propósito:** Auditoria e diagnóstico. Retorna até 500 lançamentos individuais de um período.

```
# Query ORM – equivalente SQL:  
SELECT *  
FROM dw.fato_lancamento_realizado  
WHERE EXTRACT(YEAR FROM data_referencia) = :ano  
AND EXTRACT(MONTH FROM data_referencia) = :mes  
AND (:id_empresa IS NULL OR id_empresa = :id_empresa)  
AND (:id_conta_gerencial IS NULL OR id_conta_gerencial = :id_conta_gerencial)  
LIMIT 500
```

O limite de 500 é intencional — o endpoint não é para análise em massa, mas para verificar lançamentos específicos.

5. API — Queries ORM

Queries de CRUD executadas pelo SQLAlchemy ORM. Mais simples que as analíticas, mas com lógica de negócio importante.

5.1 Orçamento — Listar

```
# api/routers/orcamento.py
db.query(FatoOrcamento)
.filter(FatoOrcamento.ano == ano, FatoOrcamento.id-versao == id-versao)
.filter(FatoOrcamento.id-empresa == id-empresa) # se informado
.filter(FatoOrcamento.id-centro-custo == id-centro-custo) # se informado
.all()
```

5.2 Orçamento — Upsert (criar ou atualizar célula)

A lógica de upsert é feita no Python (não via SQL) para retornar o objeto Pydantic correto:

```
# Verifica se já existe lançamento para mesma combinação
existente = db.query(FatoOrcamento).filter(
    FatoOrcamento.id-empresa == payload.id-empresa,
    FatoOrcamento.id-versao == payload.id-versao,
    FatoOrcamento.id-conta-gerencial == payload.id-conta-gerencial,
    FatoOrcamento.id-centro-custo == payload.id-centro-custo,
    FatoOrcamento.ano == payload.ano,
    FatoOrcamento.mes == payload.mes,
).first()

if existente:
    existente.valor = payload.valor # atualiza
    existente.observacao = payload.observacao
else:
    obj = FatoOrcamento(**payload.model_dump()) # insere
    db.add(obj)
```

Retorna 409 se a versão estiver bloqueada.

5.3 Workflow — Verificações de estado

```
# Bloqueia duplicata de workflow ativo (RASCUNHO ou ENVIADO)
db.query(WorkflowOrcamento).filter(
    WorkflowOrcamento.id-versao == payload.id-versao,
    WorkflowOrcamento.id-empresa == payload.id-empresa,
    WorkflowOrcamento.status.in_(["RASCUNHO", "ENVIADO"]),
).first()
```

```
# Validação de transição de estado
def _exige_status(wf, esperado):
    if wf.status != esperado.value:
        raise HTTPException(409, f"Status atual '{wf.status}' - esperado '{esperado.value}'")
```

Transições:

- * RASCUNHO → ENVIADO: enviar_para_revisao() — dispara e-mail em background
- * ENVIADO → APROVADO: aprovar() — bloqueia versão + e-mail
- * ENVIADO → REPROVADO: reprovar() — comentário obrigatório + e-mail

5.4 Mapeamentos — Anti-duplicata

```
# Antes de inserir mapeamento de conta, verifica se já existe ativo
db.query(MapeamentoContaSia).filter(
  MapeamentoContaSia.id_conta_sia == payload.id_conta_sia,
  MapeamentoContaSia.id_empresa == payload.id_empresa,
  MapeamentoContaSia.ativo == True,
).first()
# Se existir → HTTP 409 Conflict
```

A mesma lógica vale para MapeamentoCentroCusto.

Delete = soft delete:

```
obj.ativo = False # não remove do banco
db.commit()
```

5.5 Mapeamentos — Contas SIA disponíveis

```
db.query(DimContaSia)
  .filter(DimContaSia.codpla == codpla) # opcional
  .filter(DimContaSia.conta_nivel == nivel) # opcional
  .order_by(DimContaSia.codpla, DimContaSia.conta_class)
  .all()
```

5.6 Justificativas de variação — listar por workflow

```
db.query(JustificativaVariacao).filter(
  JustificativaVariacao.id-versao == wf.id-versao,
  JustificativaVariacao.id_empresa == wf.id_empresa,
).all()
```

6. Views Analíticas

Views criadas pelas migrations 003 e 004. Usadas pelo Metabase e disponíveis para consulta direta.

6.1 dw.v_lancamentos_detalhado — migration 003

Propósito: Desnormaliza fato_lancamento_realizado com todos os JOINS de dimensão para facilitar relatórios no Metabase.

```

SELECT
  lr.id,
  lr.sia_lancamento_id,
  dt.data AS data_referencia,
  dt.ano, dt.mes, dt.nome_mes, dt.trimestre, dt.semestre,
  e.id AS id_empresa, e.nome AS empresa,
  cs.conta_codigo AS sia_conta_codigo,
  cs.conta_class AS sia_conta_class,
  cs.conta_nome AS sia_conta_nome,
  cg.id AS id_conta_gerencial,
  cg.codigo AS conta_gerencial_codigo,
  cg.nome AS conta_gerencial_nome,
  cg.tipo AS conta_tipo,
  cc.id AS id_centro_custo, -- sempre NULL
  cc.codigo AS cc_codigo, -- sempre NULL
  cc.nome AS cc_nome, -- sempre NULL
  lr.tipo_lancamento,
  lr.valor,
  lr.valor * CASE WHEN lr.tipo_lancamento = 'D' THEN 1 ELSE -1 END AS valor_liquido,
  lr.historico,
  lr.data_carga
FROM dw.fato_lancamento_realizado lr
JOIN dw.dim_tempo dt ON dt.id = lr.id_tempo
JOIN dw.dim_empresa e ON e.id = lr.id_empresa
JOIN dw.dim_conta_sia cs ON cs.id = lr.id_conta_sia
LEFT JOIN dw.dim_conta_gerencial cg ON cg.id = lr.id_conta_gerencial -- NULL se não
mapeado
LEFT JOIN dw.dim_centro_custo cc ON cc.id = lr.id_centro_custo -- sempre NULL

```

valor_liquido: valor com sinal (+D/-C). Útil para somar diretamente sem CASE WHEN no Metabase.

6.2 dw.v_comparativo_mensal — corrigida pela migration 004

Propósito: Comparativo mensal por conta/empresa/versão para dashboards do Metabase.

A versão da migration 003 tinha o bug do CC join. A migration 004 a substituiu:

```

WITH realizado_agg AS (
  -- Agrega por (empresa, conta, ano, mes) – sem CC
  SELECT
    id_empresa,
    id_conta_gerencial,
    EXTRACT(YEAR FROM data_referencia)::int AS ano,
    EXTRACT(MONTH FROM data_referencia)::int AS mes,
    SUM(valor * CASE WHEN tipo_lancamento = 'D' THEN 1 ELSE -1 END) AS valor_realizado
  FROM dw.fato_lancamento_realizado
  WHERE id_conta_gerencial IS NOT NULL
  GROUP BY id_empresa, id_conta_gerencial,
    EXTRACT(YEAR FROM data_referencia),
    EXTRACT(MONTH FROM data_referencia)
)
SELECT
  fo.ano, fo.mes, <nome_mes>,
  v.id AS id-versao, v.nome AS versao_nome, v.tipo AS versao_tipo,
  e.id AS id_empresa, e.nome AS empresa_nome,
  cg.id AS id_conta_gerencial,
  cg.codigo AS conta_codigo, cg.nome AS conta_nome,
  cg.tipo AS conta_tipo, cg.natureza AS conta_natureza,
  SUM(fo.valor) AS valor_orcado,
  COALESCE(MAX(r.valor_realizado), 0) AS valor_realizado,
  COALESCE(MAX(r.valor_realizado), 0) - SUM(fo.valor) AS variacao_absoluta,
  ROUND(((COALESCE(MAX(r.valor_realizado), 0) - SUM(fo.valor)) / ABS(SUM(fo.valor)) * 100),
  2)
  AS variacao_percentual
FROM dw.fato_orcamento fo
JOIN dw.dim-versao-orcamento v ON v.id = fo.id-versao
JOIN dw.dim_empresa e ON e.id = fo.id_empresa
JOIN dw.dim_conta_gerencial cg ON cg.id = fo.id_conta_gerencial
LEFT JOIN realizado_agg r
  ON r.id_empresa = fo.id_empresa
  AND r.id_conta_gerencial = fo.id_conta_gerencial
  AND r.ano = fo.ano AND r.mes = fo.mes
GROUP BY fo.ano, fo.mes, v.id, v.nome, v.tipo,
  e.id, e.nome, cg.id, cg.codigo, cg.nome, cg.tipo, cg.natureza

```

Diferença para a query da API: A view mantém `id_empresa` no GROUP BY do CTE e no JOIN externo (para que o Metabase possa filtrar por empresa). A API endpoint remove empresa do CTE GROUP BY para suportar filtro pontual sem multi-empresa.

6.3 dw.v_evolucao_mensal — migration 003

Propósito: Totais mensais de realizado por tipo de conta. Útil para gráficos de tendência no Metabase.

```

SELECT
  dt.ano, dt.mes, dt.nome_mes, dt.trimestre,
  e.id AS id_empresa, e.nome AS empresa,
  cg.tipo AS conta_tipo,
  SUM(lr.valor * CASE WHEN lr.tipo_lancamento = 'D' THEN 1 ELSE -1 END) AS valor_realizado,
  COUNT(DISTINCT lr.sia_lancamento_id) AS qtd_lancamentos
FROM dw.fato_lancamento_realizado lr
JOIN dw.dim-tempo dt ON dt.id = lr.id-tempo
JOIN dw.dim_empresa e ON e.id = lr.id_empresa
JOIN dw.dim_conta_gerencial cg ON cg.id = lr.id_conta_gerencial
WHERE lr.id_conta_gerencial IS NOT NULL
GROUP BY dt.ano, dt.mes, dt.nome_mes, dt.trimestre, e.id, e.nome, cg.tipo

```

6.4 dw.v_dre_anual — migration 003

Propósito: DRE completa com hierarquia, orçado e realizado por ano/versão/empresa.

```
WITH realizado_anual AS (  
  SELECT id_conta_gerencia, id_empresa,  
    EXTRACT(YEAR FROM data_referencia)::int AS ano,  
    SUM(valor * CASE WHEN tipo_lancamento = 'D' THEN 1 ELSE -1 END) AS valor_realizado  
  FROM dw.fato_lancamento_realizado WHERE id_conta_gerencia IS NOT NULL  
  GROUP BY id_conta_gerencia, id_empresa, EXTRACT(YEAR FROM data_referencia)  
)  
,  
orcado_anual AS (  
  SELECT id_conta_gerencia, id_empresa, id-versao, ano,  
    SUM(valor) AS valor_orcado  
  FROM dw.fato_orcamento  
  GROUP BY id_conta_gerencia, id_empresa, id-versao, ano  
)  
SELECT  
  cg.id, cg.codigo, cg.nome, cg.tipo, cg.natureza, cg.nivel,  
  cg.id_pai, pai.codigo AS codigo_pai, pai.nome AS nome_pai,  
  v.id AS id-versao, v.nome AS versao_nome, v.ano,  
  e.id AS id_empresa, e.nome AS empresa_nome,  
  COALESCE(o.valor_orcado, 0) AS valor_orcado,  
  COALESCE(r.valor_realizado, 0) AS valor_realizado,  
  COALESCE(r.valor_realizado, 0) - COALESCE(o.valor_orcado, 0) AS variacao_absoluta,  
  ROUND(((COALESCE(r.valor_realizado, 0) - COALESCE(o.valor_orcado, 0))  
    / ABS(COALESCE(o.valor_orcado, 0)) * 100), 2) AS variacao_percentual  
  FROM dw.dim_conta_gerencia cg  
  LEFT JOIN dw.dim_conta_gerencia pai ON pai.id = cg.id_pai  
  CROSS JOIN dw.dim-versao_orcamento v -- todas as versões  
  CROSS JOIN dw.dim_empresa e -- todas as empresas  
  LEFT JOIN orcado_anual o ON o.id_conta_gerencia = cg.id AND o.id_empresa = e.id AND  
    o.id-versao = v.id  
  LEFT JOIN realizado_anual r ON r.id_conta_gerencia = cg.id AND r.id_empresa = e.id AND  
    r.ano = v.ano  
  WHERE cg.ativa = true AND e.ativa = true  
  AND (o.valor_orcado IS NOT NULL OR r.valor_realizado IS NOT NULL)
```

CROSS JOIN: A view cruza todas as contas × versões × empresas e filtra apenas onde há dados (orçado ou realizado). Isso garante que toda combinação com dados apareça, mesmo se faltar um dos lados.

6.5 dw.v_workflow_resumo — migration 003

Propósito: Status de aprovação desnormalizado para dashboards.

```

SELECT
w.id, w.status,
v.id AS id-versao, v.nome AS versao_nome, v.ano AS versao_ano,
v.tipo AS versao_tipo, v.bloqueada AS versao_bloqueada,
e.id AS id_empresa, e.nome AS empresa_nome,
w.criado_por, w.enviado_por, w.aprovado_por, w.reprovado_por,
w.data_envio, w.data_decisao, w.comentario,
w.criado_em, w.atualizado_em,
CASE
WHEN w.data_decisao IS NOT NULL AND w.data_envio IS NOT NULL
THEN ROUND(EXTRACT(EPOCH FROM (w.data_decisao - w.data_envio)) / 3600, 1)
ELSE NULL
END AS horas_para_decisao
FROM dw.workflow_orcamento w
JOIN dw.dim-versao-orcamento v ON v.id = w.id-versao
JOIN dw.dim_empresa e ON e.id = w.id_empresa

```

horas_para_decisao: Calculado em horas com 1 casa decimal. `EXTRACT(EPOCH FROM ...)` retorna diferença em segundos; dividir por 3600 converte para horas. Útil para medir tempo médio de aprovação por período.

Resumo — Mapa de Queries por Fluxo

```

USUÁRIO ACESSA DASHBOARD
|
+-- GET /comparativo/{ano}/{id-versao}
| |-- CTE realizado_agg: fato_lancamento_realizado
| |-- fato_orcamento (filtro versao+empresa+CC)
| |-- LEFT JOIN por (conta, ano, mes)
|
+-- GET /dre/{ano}/{id-versao}
| |-- CTE orcado: fato_orcamento GROUP BY conta
| |-- CTE realizado: fato_lancamento_realizado GROUP BY conta
| |-- dim_conta_gerecncial (todas ativas) LEFT JOIN ambos CTES
|
|-- GET /lancamentos/{YYYY-MM} (auditoria)
|-- fato_lancamento_realizado WHERE ano+mes LIMIT 500

PIPELINE ETL (mensal)
|
+-- Passo 1: GER_EMPRESAS → dim_empresa (upsert por codemp)
+-- Passo 2: CTB_CONTAS → dim_conta_sia (upsert por codpla+codigo)
+-- Passo 3: CTB_MOVIMENTOS → resolver FKs → fato_lancamento_realizado
| |-- resolver_ids_empresa: dim_empresa WHERE codemp = ANY(...)
| |-- resolver_ids_tempo: dim_tempo WHERE data = ANY(...)
| |-- resolver_ids_conta_sia: DISTINCT ON conta_codigo ORDER BY codpla DESC
| |-- resolver_mapeamentos_conta: JOIN mapeamento + dim_conta_sia
| |-- upsert fato ON CONFLICT (sia_lancamento_id) DO UPDATE
|-- (dim_tempo gerado localmente, sem query ao SIA)

```


Decisoes Tecnicas

Architecture Decision Records (ADRs)

Decisões Técnicas — Architecture Decision Records

Registro das decisões de arquitetura tomadas durante o desenvolvimento da plataforma FP&A, com contexto, alternativas consideradas e justificativas.

ADR-001 — Decimal em vez de Float para valores monetários

Status: Aceito Data: 2025-01

Contexto

Valores monetários precisam de precisão exata. `float` em Python usa IEEE 754 binary floating-point, que não representa exatamente valores como `0.10` ($0.1 + 0.2 \neq 0.3$). Para um sistema financeiro, isso é inaceitável.

Decisão

- * Python: sempre `decimal.Decimal`, nunca `float`
- * PostgreSQL: sempre `NUMERIC(15,2)`, nunca `FLOAT`, `REAL` ou `DOUBLE PRECISION`
- * SQLAlchemy: coluna `NUMERIC(15,2)` mapeada automaticamente para `Decimal`
- * Pydantic: serializa `Decimal` como string numérica (ex: `"15000.00"`)
- * Frontend: `Number(valor)` apenas no momento de exibição/cálculo; armazenado como string

Alternativas descartadas

- * `float` — precisão insuficiente para finanças
- * `int` (centavos) — mais difícil de debugar, conversões implícitas em todo lugar

Consequências

Toda função que manipula valor monetário deve importar e usar `Decimal`. Qualquer `float` no código é um bug.

ADR-002 — SIA é somente leitura

Status: Aceito Data: 2025-01

Contexto

O SIA (System SIA, ERP em Firebird) é o sistema de registro oficial da empresa. Qualquer escrita incorreta pode corromper dados contábeis e fiscais, com impacto legal.

Decisão

A conexão ao Firebird é configurada com `readonly=True`. O ETL só executa `SELECT`. Nenhum `INSERT`, `UPDATE` ou `DELETE` é permitido no SIA.

Como isso é garantido

```
# extractor.py
self._conn = pyodbc.connect(conn_str, autocommit=False, readonly=True)
```

O `readonly=True` do `pyodbc` sinaliza ao driver que é uma conexão somente leitura. Qualquer tentativa de escrita levantaria exceção no nível do driver.

Consequências

Todo dado de planejamento (orçamento, mapeamentos, workflow) fica exclusivamente no PostgreSQL DW, nunca no SIA.

ADR-003 — `MOV_CECT = NULL` — nenhum lançamento tem centro de custo

Status: Aceito Data: 2026-05

Contexto

Durante a análise dos dados, foi constatado via MCP Firebird que **100% dos lançamentos em CTB_MOVIMENTOS têm `MOV_CECT = NULL`** para as empresas Philozon. Isso significa que o SIA não está sendo usado para registrar o CC nos lançamentos contábeis — os lançamentos são apenas por conta.

Investigação

Consultado diretamente via:

```
SELECT COUNT(*) FROM CTB_MOVIMENTOS WHERE MOV_CODEMP = 1 AND MOV_CECT IS NOT NULL
-- Resultado: 0
```

Impacto

A query original do comparativo fazia:

```
LEFT JOIN fato_lancamento_realizado lr ON lr.id_centro_custo = fo.id_centro_custo
```

Como `id_centro_custo` é sempre `NULL` e `NULL ≠ NULL` em SQL, o join **nunca casava** — `valor_realizado` retornava zero para todos os registros.

Decisão

A estratégia correta para o comparativo Realizado × Orçado:

- > **Realizado**: agregar por (`conta_gerencia`, `empresa`, `ano`, `mes`) — ignorar CC
- > **Orçado**: somar todos os CCs da mesma conta-mes (`SUM(fo.valor)` sem filtrar CC)
- > **JOIN**: apenas por (`conta_gerencia`, `ano`, `mes`) (+ `empresa` quando filtrado)
- > **Output**: `centro_custo_codigo = NULL`, `centro_custo_nome = NULL` — documentado na API

Implementação

- * `api/routers/comparativo.py`: CTE `realizado_agg` sem CC, `GROUP BY` sem CC
- * `migrations/004_fix_comparativo_cc.py`: corrige `v_comparativo_mensal`
- * `api/schemas/comparativo.py`: campos CC declarados como `str` | `None = None`

* frontend/src/services/api.ts: centro_custo_nome: string | null

Nota sobre o filtro CC no orçado

O filtro `id_centro_custo` na API do comparativo ainda funciona para o **orçado** (filtra quais CCs do orçamento entram na soma). O realizado não tem CC — é sempre o total da conta.

Consequências

O campo `centro_custo_nome` é sempre `null` na resposta do comparativo. Isso é documentado e esperado. Se o SIA vier a registrar CCs no futuro, a estratégia precisaria ser revisada — mas a view `v_comparativo_mensal` na migration 004 teria que ser atualizada com uma nova migration.

ADR-004 — Multi-empresa: filtro `id_empresa` deve ser aplicado nos dois lados

Status: Aceito Data: 2026-05

Contexto

A plataforma suporta múltiplas empresas (`dim_empresa`). Quando o frontend filtra por empresa, é crítico que o filtro seja aplicado **tanto no realizado quanto no orçado**.

O bug

A versão inicial do `realizado_agg` agrupava por (`conta, empresa, mes`). O JOIN externo agrupava por (`conta, mes`) sem empresa. Com múltiplas empresas:

- * `empresa=1: realizado=300`
- * `empresa=3: realizado=80`
- * `MAX(r.valor_realizado)` retornava 300 em vez de 380 (se consolidado) ou 300/80 separados

A correção

Remover `id_empresa` do GROUP BY do CTE e aplicar o filtro dentro do WHERE do CTE:

```
WITH realizado_agg AS (  
  SELECT id_conta_gerencial, ano, mes,  
    SUM(valor * ...) AS valor_realizado  
  FROM dw.fato_lancamento_realizado  
  WHERE id_conta_gerencial IS NOT NULL  
  AND (:id_empresa IS NULL OR id_empresa = :id_empresa) -- filtro aqui  
  GROUP BY id_conta_gerencial, ano, mes -- sem empresa  
)
```

- * Com `id_empresa = X`: o CTE agrega apenas a empresa X → uma linha por (`conta, mes`) → correto
- * Sem filtro (`id_empresa IS NULL`): o CTE agrega todas as empresas → visão consolidada → correto

Consequências

O filtro `id_empresa` deve sempre ser passado nas chamadas do frontend. O hook `useEmpresaAtiva()` resolve automaticamente o `id` correto a partir de `VITE_EMPRESA_CODEMP`. Nunca hardcodar `id_empresa = 1`.

ADR-005 — `VITE_EMPRESA_CODEMP` em vez de hardcodar `id_empresa`

Status: Aceito Data: 2026-05

Contexto

O campo `id` da tabela `dim_empresa` é autoincrement do DW, não previsível. O código SIA da empresa (`codemp`) é estável e definido no ERP. O frontend precisa de um jeito de saber qual empresa usar sem hardcodar um ID interno.

Decisão

`VITE_EMPRESA_CODEMP` é definido no `.env` do frontend com o `EMP_COD` da empresa principal (ex: 1 para Philozon).

O hook `useEmpresaAtiva()` busca GET `/empresas/` na inicialização e encontra a empresa cujo `codemp` `=== VITE_EMPRESA_CODEMP`. Retorna o `id` do DW correspondente.

```
// hooks/useDimensoes.ts
export function useEmpresaAtiva() {
  const codemp = Number(import.meta.env.VITE_EMPRESA_CODEMP);
  const { data: empresas = [] } = useEmpresas();
  const empresa = empresas.find((e) => e.codemp === codemp);
  return { idEmpresa: empresa?.id ?? null, empresa };
}
```

Alternativas descartadas

- * Hardcodar `id_empresa = 1` — frágil: o ID pode mudar entre ambientes
- * Passar empresa via URL — complexidade desnecessária para uso monoempresa

Consequências

Todo componente que faz chamadas filtradas por empresa deve usar `useEmpresaAtiva()`. Nenhum valor de `id_empresa` deve ser hardcodado no frontend.

ADR-006 — Sem autenticação na versão atual

Status: Aceito (revisitar em produção multi-usuário) Data: 2025-01

Contexto

A plataforma é de uso interno, acessível apenas na rede da empresa. Implementar autenticação completa (OAuth, JWT, RBAC) adicionaria complexidade significativa.

Decisão

A API não implementa autenticação. Em produção:

- * Nginx protege o acesso com autenticação básica (HTTP Basic Auth) ou a rede está atrás de VPN
- * Todos os endpoints são acessíveis para qualquer usuário autenticado via rede

Campos "criado_por" / "aprovado_por"

Os campos de auditoria (`criado_por`, `enviado_por`, `aprovado_por`) são strings livres enviadas pelo frontend — não há validação de identidade. São apenas para rastreabilidade informativa.

Quando revisar

Se a plataforma crescer para uso multi-usuário com controle de acesso granular (ex: gestor pode aprovar, analista só lança), implementar JWT + RBAC.

ADR-007 — Plano gerencial separado do contábil SIA

Status: Aceito Data: 2025-01

Contexto

O plano de contas do SIA (CTB_CONTAS) é contábil — segue as regras do SPED e da contabilidade fiscal brasileira. Para análise gerencial, o ideal é um plano simplificado e orientado para tomada de decisão.

Decisão

Dois planos paralelos:

- * `dim_conta_sia` — espelho do SIA (contábil, leitura)
- * `dim_conta_gerencial` — cadastro interno (gerencial, read/write via API)

A conexão entre os dois é feita pela tabela `mapeamento_conta_sia_gerencial`:

```
CTB_CONTAS (SIA) → dim_conta_sia → mapeamento → dim_conta_gerencial
```

O ETL usa esse mapeamento para popular `id_conta_gerencial` em `fato_lancamento_realizado`.

Consequências

Contas não mapeadas têm `id_conta_gerencial = NULL` no fato. Elas aparecem nos lançamentos mas não no comparativo gerencial. O usuário deve cadastrar os mapeamentos antes de usar o comparativo.

ADR-008 — Versionamento do orçamento

Status: Aceito Data: 2025-01

Contexto

Um orçamento anual passa por múltiplas revisões ao longo do ano (original, revisões, forecast trimestral). É necessário manter histórico de cada versão para comparação.

Decisão

`dim-versao-orcamento` com `tipo` ∈ {ORIGINAL, REVISAO, FORECAST}. Cada versão tem uma flag bloqueada que é ativada ao aprovar via workflow, impedindo edições retroativas.

O comparativo sempre recebe `id-versao` explicitamente — nunca assume "a versão mais recente".

ADR-009 — SQL raw para queries analíticas complexas

Status: Aceito Data: 2025-01

Contexto

Queries do comparativo, DRE e lançamentos envolvem CTEs, CASE WHEN, EXTRACT, GROUP BY complexos. O ORM do SQLAlchemy dificulta expressá-los de forma legível.

Decisão

- * CRUD simples (dimensões, workflow) → SQLAlchemy ORM
- * Queries analíticas (comparativo, DRE, lançamentos) → `sqlalchemy.text()` com SQL raw

```
# Para CRUD simples
empresa = db.get(DimEmpresa, id)

# Para queries analíticas
rows = db.execute(text("WITH ... SELECT ..."), {"param": value}).mappings().all()
```

Consequências

SQL analítico deve ser testado diretamente no PostgreSQL antes de integrar ao router. Parâmetros devem usar `:nome` (não `?`) — SQLAlchemy converte para o driver correto.

ADR-010 — `fato_lancamento_realizado` vs views agregadas

Status: Aceito **Data:** 2025-01

Contexto

Poderíamos agregar os lançamentos diretamente no ETL e carregar apenas totais mensais. Isso simplificaria as queries, mas perderia granularidade.

Decisão

Carregar lançamentos individuais em `fato_lancamento_realizado` e agregar sob demanda via SQL e views. As views `v_comparativo_mensal`, `v_evolucao_mensal`, etc. materializam as agregações comuns para o Metabase.

Consequências

- * `fato_lancamento_realizado` pode crescer rapidamente (centenas de milhares de linhas por ano)
- * Queries analíticas precisam de GROUP BY — as views do Metabase são a solução
- * Para auditoria, o endpoint `GET /lançamentos/{mes_referencia}` retorna até 500 lançamentos individuais